

Boolean Logic & Elementary Gates

Lab 1 — Nand2Tetris Project 1

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

August 21, 2026

Aim. *To design and simulate the elementary logic gates of a computer system — starting from a single primitive Nand gate — using the Hardware Description Language (HDL) and the Nand2Tetris Hardware Simulator.*

1.1 Learning Outcomes

On completing this lab, a student will be able to:

- State the functional (truth-table) specification of **Not**, **And**, **Or**, **Xor**, **Mux** and **DMux** gates.
- Compose complex gates purely from **Nand**, using gate-level composition in HDL.
- Extend single-bit gates to 16-bit buses and multi-way variants.
- Verify a chip’s correctness independently of its internal implementation, using supplied test scripts.

1.2 Pre-Lab Reading

Pre-Lab Reading. **Boolean algebra and gate-level logic** — this lab’s prerequisite theory (AND/OR/NOT/XOR truth tables, De Morgan’s laws, Boolean minimization) is not covered by any CS401 lecture; it is assumed prerequisite knowledge from Digital Logic Design (see `syllabi/CS401.md`’s Prerequisites). For a page-verified refresher:

- Patterson & Hennessy, *Computer Organization and Design, ARM Edition*, Appendix A — §A.2 *Gates, Truth Tables, and Logic Equations* (p. A-4), §A.3 *Combinational Logic* (p. A-9), §A.4 *Using a Hardware Description Language* (p. A-20). Page-cited against this course’s PattersonHennessy2017 textbook index.
- Nisan & Schocken, *The Elements of Computing Systems* (MIT Press), Chapter 1 — *Boolean Logic*. Not indexed here (chapter-level citation only, per the textbook-grounding rule).
- The *HDL Survival Guide* (Nand2Tetris, Appendix 2, nand2tetris.org) — the syntax reference for this lab’s `.hdl` files.

Also review: the concept of a **primitive** gate and the **functional completeness** of **Nand** (every Boolean function can be built from **Nand** alone) — this is the single idea the whole lab demonstrates.

1.3 Toolchain Setup

1. Install a Java Runtime Environment (JRE), version 8 or later, if not already present.
2. Download the Nand2Tetris software suite (free, nand2tetris.org) — this lab uses the **Hardware Simulator** only; the CPU Emulator and Assembler are needed from Lab 4 onward.
3. Extract the suite and locate the `projects/01/` folder — it contains one subfolder per chip, pre-populated with a `ChipName.hdl` stub (to be completed) and the supplied `ChipName.tst` / `ChipName.cmp` test/comparison scripts. These scripts are standard Nand2Tetris project files and are *not* redistributed here — obtain them from the official suite.
4. Open the Hardware Simulator, load a chip's `.tst` script (FILE → LOAD SCRIPT), and run it (SIMULATION → RUN). A correct chip prints `End of script --- comparison ended successfully` in the script's output pane; a wrong one reports the first mismatching row.
5. For a full walkthrough of the Hardware Simulator's interface (loading a chip, interactive pin testing, exploring PARTS, running scripts, debugging with breakpoints), see the official *Hardware Simulator Tutorial* (Nand2Tetris, nand2tetris.org).

Every chip's `.hdl` file must follow the CHIP/IN/OUT/PARTS syntax specified in Appendix A of the Nand2Tetris book: a part is invoked as `PartName(pin=value, ...)`, where each `value` is either another pin/bus in the current chip or a new internal pin created on first use; a bus pin `b[16]` is indexed `b[i]` for a single bit or sliced `b[lo..hi]` for a sub-range. Only `Nand` is a true primitive (BUILTIN `Nand`;) — every other chip in this lab must be built from `Nand` and/or previously-built chips.

1.4 Tasks

Implement the following 15 chips **in order** — each later chip may reuse any chip already built earlier in the list, and several depend on it (e.g. `Not16` is 16 parallel copies of `Not`). Only `Nand` itself is a supplied primitive; every other chip below must be composed from `Nand` and/or previously-built chips — never from a built-in gate.

1. `Not(in) → out` — $out = \overline{in}$. Hint: a `Nand` gate with both inputs tied to the same signal is a `Not` gate.
2. `And(a,b) → out` — $out = a \cdot b$. Hint: `Nand` then `Not` the result.
3. `Or(a,b) → out` — $out = a + b$. Hint: De Morgan — `Not` each input, `Nand` the results.
4. `Xor(a,b) → out` — $out = a \oplus b$. Hint: $a \oplus b = (a \cdot \bar{b}) + (\bar{a} \cdot b)$; needs `And`, `Not`, `Or`.
5. `Mux(a,b,sel) → out` — $out = a$ if $sel = 0$, else $out = b$. A 2-to-1 multiplexer.
6. `DMux(in,sel) → (a,b)` — routes `in` to output `a` if $sel = 0$, to `b` if $sel = 1$; the unselected output is 0. A 1-to-2 demultiplexer, the mirror image of `Mux`.
7. `Not16(in[16]) → out[16]` — 16 independent `Not` gates, one per bit.
8. `And16(a[16],b[16]) → out[16]` — 16 independent `And` gates.

9. `Or16(a[16],b[16]) → out[16]` — 16 independent Or gates.
10. `Mux16(a[16],b[16],sel) → out[16]` — one `sel` bit chooses between two 16-bit buses, bit-by-bit.
11. `Or8Way(in[8]) → out` — $out = in_0 + in_1 + \dots + in_7$ (OR-reduce 8 single bits to 1). Hint: a 7-deep Or tree, or a chain.
12. `Mux4Way16(a,b,c,d[16] each, sel[2]) → out[16]` — selects one of 4 sixteen-bit inputs via a 2-bit selector. Hint: compose from Mux16.
13. `Mux8Way16(a..h[16] each, sel[3]) → out[16]` — selects one of 8 sixteen-bit inputs via a 3-bit selector. Hint: two Mux4Way16s plus one more Mux16.
14. `DMux4Way(in,sel[2]) → (a,b,c,d)` — routes `in` to exactly one of 4 outputs.
15. `DMux8Way(in,sel[3]) → (a,b,c,d,e,f,g,h)` — routes `in` to exactly one of 8 outputs.

Load each chip's supplied `.tst` script in the Hardware Simulator and confirm the comparison against the `.cmp` file succeeds before moving to the next chip in the list.

1.5 Figures

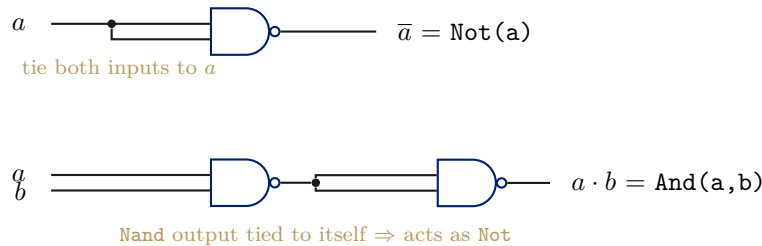


Figure 1: Not and And composed entirely from the Nand primitive — the pattern every other chip in this lab extends. Not(a) ties both Nand inputs to a ; And(a,b) feeds Nand(a,b) into a second Nand wired as Not.

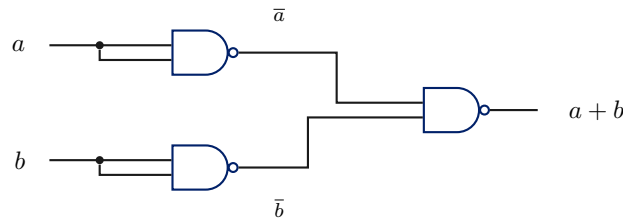


Figure 2: Or from three Nand gates (De Morgan): $a + b = \overline{\bar{a} \cdot \bar{b}}$. Each input Nand is wired with both inputs tied together, acting as an inverter; the third Nand combines the two negations.

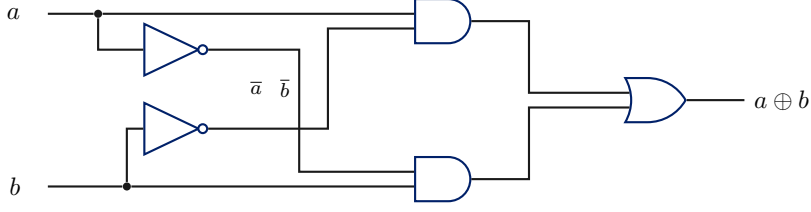


Figure 3: **Xor** from **Not/And/Or**: $a \oplus b = (a \cdot \bar{b}) + (\bar{a} \cdot b)$. Two inverters produce $\bar{a}, \bar{b}$; the two **And** gates form the product terms; the **Or** sums them.

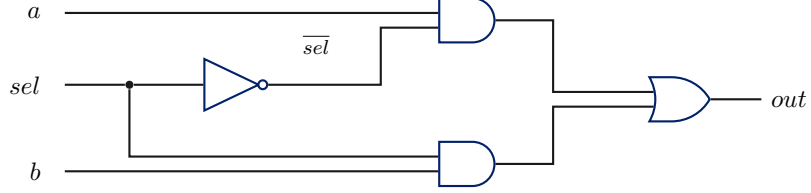


Figure 4: **Mux**: $out = (a \cdot \overline{sel}) + (b \cdot sel)$. The inverter makes $\overline{sel}$; the top **And** passes a when $sel=0$, the bottom passes b when $sel=1$; the **Or** merges them.

1.6 Truth Tables & Boolean Expressions (step by step)

Every chip in this lab is a pure Boolean function of its inputs. The tables below give each chip's truth table; the equations show the step-by-step Nand-only construction used in the Figures.

Step-by-step Nand-only derivations.

Not and And (Figure 1):

$$\bar{a} = \text{NAND}(a, a) = \overline{a \cdot a} = \bar{a}$$

$$a \cdot b = \text{NAND}(\text{NAND}(a, b), \text{NAND}(a, b)) = \overline{\overline{a \cdot b}} = a \cdot b$$

Or via De Morgan (Figure 2):

$$\bar{a} = \text{NAND}(a, a), \quad \bar{b} = \text{NAND}(b, b)$$

$$a + b = \text{NAND}(\bar{a}, \bar{b}) = \overline{\bar{a} \cdot \bar{b}} = a + b \quad (\text{De Morgan: } \overline{\bar{a} \cdot \bar{b}} = a + b)$$

Xor as sum of minterms (Figure 3):

$$\begin{aligned} a \oplus b &= (a \cdot \bar{b}) + (\bar{a} \cdot b) \\ &= \text{OR}(\text{AND}(a, \text{NOT } b), \text{AND}(\text{NOT } a, b)) \end{aligned}$$

Mux (Figure 4):

$$out = (a \cdot \overline{sel}) + (b \cdot sel)$$

DMux (Figure 5):

$$a = in \cdot \overline{sel}, \quad b = in \cdot sel$$

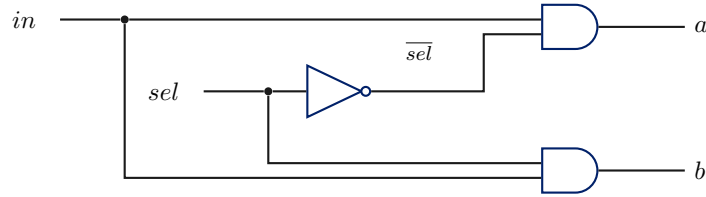


Figure 5: **DMux**: $a = in \cdot \overline{sel}$, $b = in \cdot sel$. One inverter and two **And** gates; the unselected output is forced to 0 because its **And** input is gated off.

Inputs		Not	And	Or	Xor
a	b	$\bar{a}$	$a \cdot b$	$a + b$	$a \oplus b$
0	0	1	0	0	0
0	1	1	0	1	1
1	0	0	0	1	1
1	1	0	1	1	0

Table 1: Truth tables for the four single-bit gates (one row per input combination).

1.7 Observation / Journal Table

Record, for each chip, the gates used in your composition and whether it passed its supplied test script.

Journal instruction (separate from the table above): for *two* chips of your choice, hand-draw the gate-level schematic and write out the full truth table in your lab journal (the syllabus's own Deliverable 2).

1.8 Viva-Voce Questions

1. Why is **Nand** *functionally complete* — i.e. why can every Boolean function be built from **Nand** alone?
2. Derive **Or(a,b)** from **Nand** using De Morgan's law. Which gates does your derivation use, and why can't you use a plain **Or** primitive?
3. Trace **Mux(1,0,1)** by hand through your composition. What is out?
4. Why does **Or8Way** need 7 **Or** gates for 8 inputs, not 8?
5. What would go wrong if **DMux**'s unselected output were left floating instead of driven to 0?

1.9 Rubric

Reproduced verbatim from [syllabi/CS401_Lab_Syllabus_Nand2Tetris.pdf](#).

Mux inputs			out				
<i>a</i>	<i>b</i>	<i>sel</i>		DMux inputs		outputs	
				<i>in</i>	<i>sel</i>	<i>a</i>	<i>b</i>
0	0	0	0	0	0	0	0
0	1	0	0	1	0	1	0
1	0	0	1	0	1	0	0
1	1	0	1	1	1	0	1
0	0	1	0				
0	1	1	1				
1	0	1	0				
1	1	1	1				

Table 2: Truth tables for Mux ($out = sel ? b : a$) and DMux (route in to a if $sel=0$, else to b).

Chip	Gates used (internal composition)	Pass / Fail
Not		
And		
Or		
Xor		
Mux		
DMux		
Not16		
And16		
Or16		
Mux16		
Or8Way		
Mux4Way16		
Mux8Way16		
DMux4Way		
DMux8Way		

Table 3: Lab 1 observation table — one row per chip.

Criterion	Marks
All 15 chips pass their test scripts without error	5
Correct, minimal gate-level composition (no redundant gates)	2
Journal: truth tables and schematics complete and accurate	2
Viva: can explain why Nand is functionally complete	1
Total	10